

Solution sketch

Fall 2025

General guidelines:

- For computations, when a numerical answer is clearly requested: It is fine if the students clearly and unequivocally show how they arrive at a number. E.g., writing " $2 * (8 + 5)$ " instead of "26" is fine and would give full marks. Writing a correct formula without showing how to plug in the given numbers is not as good as a numerical answer, when a numerical answer is requested: The student has then not fully demonstrated that they understand what the variables actually represent or how to apply the formula in practice. Also, just giving a final answer (even a correct one) without providing any insights into how they arrive at that answer, is not great: The student has then not demonstrated any reasoning and might just be guessing.
- For textual explanations: The best answers are not only correct and complete (i.e., answer the entire question), but are presented with clarity and precision, and should leave no doubt that the student understands the subject. Answers where the reader is left to fill in some blanks or has to creatively interpret what is meant, are not really great. Neither are vague and sloppy and imprecise answers, or rambling "shotgun" answers that try to score points by mentioning a lot of tangentially related facts that don't really address what is being asked.
- Several questions consist of multiple parts, e.g., you can be asked both how an algorithm works and how the algorithm is used in a search engine. The best answers address all parts of such multi-part questions. Note that some questions very explicitly ask for (and require) a discussion, not a description.

Nearest neighbors I

- (a) True. (Well, at least with Euclidean distance and no distance weighting.) The decision boundaries can be complex and non-linear, but are made up of pieces of the linear hyperplanes that separate the closest neighboring points belonging to different classes. Cf. Voronoi diagrams.
- (b) False. For $k = 1$, every training point is classified correctly by itself, so training error is zero. As k increases, a point's predicted class depends on its neighbors, some of which may belong to other classes, hence training error can increase. However, increasing k typically reduces variance and may improve generalization error on test data.
- (c) True. It relies on distance computations, so if features are measured on different scales those with larger numerical ranges dominate the distance metric. Standardizing or normalizing (e.g., zero mean and unit variance, or unit length) ensures each feature contributes comparably. Thus scaling can alter the neighborhood structure and change predictions.

- (d) False. The test set should be used only for final evaluation, not for model selection. The correct practice is to choose k using a validation set or cross-validation on the training data. Using the test set for tuning leaks information and leads to optimistically biased performance estimates.
- (e) False. (Well, the statement could be said to be true if the ties are for the majority class, but not if the ties are for a minority class. Thumbs up to any students that discuss this.) As a counter-example, consider $k = 5$, three classes, and the class counts 3, 1, and 1.

Nearest neighbors II

See, e.g., [here](#). An ideal discussion should present both what the core data structures represent (e.g., for tree-based algorithms, how are we branching, or, for graph-based algorithms, what do the nodes and edges in the graph mean?) in addition to how they are created indexing-time and consulted query-time.

Inverted index

- (a) For example, in brief:
- (i) By document identifier [see [here](#)]. Facilitates DAAT traversal and Boolean merge operations. Supports efficient concurrent processing of multiple lists.
 - (ii) By a static quality score [see 7.1.4 [here](#)]. Enables early stopping during query evaluation, as documents most likely to be relevant appear first. Also supports DAAT traversal with scoring optimizations.
 - (iii) By contribution to relevance score, i.e., impact-ordered [see 7.1.5 [here](#)], Implies TAAT evaluation. Enables early termination in scoring algorithms since the postings that contribute most to a document's score appear at the head of the list.
- (b) For example, in brief:
- (i) Variable-byte coding. Easy to implement and decode. Achieves moderate compression. Good trade-off between space and speed.
 - (ii) Gamma coding. Very compact for small integers, useful when gaps are small. Not byte-aligned.
 - (iii) Simple-9 coding. High compression for small gaps. Fast decoding, improving query performance. Simple to implement and balances speed with space efficiency.
- (c) Note that storing a purely document-level property (e.g., static quality score, or document length) in a posting is very wasteful and not a great idea, as the value is then needlessly replicated across every posting and updates become problematic. For example, in brief:

- (i) Term frequency. Improves relevance ranking (e.g., in TF-IDF and variants such as BM25.) Documents where the term occurs more frequently are considered more relevant.
- (ii) Positions of the term in the document. Enables phrase queries and proximity queries. Allows multi-term queries like *machine learning* to match only documents where the terms occur together or near each other. Enables proximity scoring.
- (iii) Field/context information. Helps refine relevance scoring, and allows fielded searches. For example, knowing which field(s) or context(s) in the document (e.g., title, abstract, body, footnote) the term appears in allows weighted scoring.

Tolerant retrieval

- (a) See, e.g., Section 3.3.3 [here](#). Should mention what an edit operation is, and also that it is the *minimal* number of edit operations needed to rewrite one into the other.
- (b) For example, “book” and “back”. To show that the edit distance is exactly 2, we need to show that (i) we can in fact rewrite one into the other with 2 edits [e.g., two substitutions in this example], and (ii) that 2 is actually minimal, i.e., that it’s not possible to rewrite one into the other with just 1 edit [insertion, deletion, substitution, possibly also transposition if we’re doing Levenshtein-Damerau distance.] Both (i) and (ii) are required to show. Item (i) can be shown by giving the two actual edit operations, and item (ii) can be shown by, for any edit operation *op*, exhaustively demonstrating that applying *op* once to the source string does not result in the target string. Providing a correctly and completely filled-out edit table between the source and target strings is another way of doing both (i) and (ii) in one swoop.
- (c) See, e.g., [here](#) and Section 7.2.4 [here](#), and [here](#). For example, in brief (and omitting good explanations):
 - (i) Edit distance. E.g., for spell-checking of queries.
 - (ii) *k*-gram indices. E.g., for increased recall with inexact matches.
 - (iii) Permuterm indices. E.g., for supporting wildcard queries.
 - (iv) Embeddings. E.g., for semantic matching.
 - (v) Other. E.g., string manipulations like compounding, stemming, or phonetic hashing that impact matching.
- (d) See, e.g., [here](#) and [here](#).

Statistical properties of terms

- (a) Estimates:
 - (i) $M = 10 \times \sqrt{1,000,000} = 10,000$
 - (ii) $f_5 = f_1 / 5 = 50,000 / 5 = 10,000$. (There’s also a proportionality constant here that we’ve ignored [or, set to 1] here, thumbs up to any students that comment on

this.)

- (b) We have a vocabulary size of 10,000 words [see (i) above] and a corpus size of 1,000,000 words. Using the given reasoning we have that the top 10% most frequent words (1,000 words) account for 90% of the corpus (900,000 words), and that the remaining 90% (9,000 words) defines a long tail that accounts for 10% of the corpus (100,000 words).
- (i) Words occurring only once: $\frac{2}{3} \times 9,000 = 6,000$
 - (ii) Words occurring exactly twice: $\frac{1}{3} \times 9,000 = 3,000$

Linkorama

- (a) See, e.g., [here](#) and [here](#). We first compute the transition probability matrix from just the adjacency list, ignoring teleportation:

$$M = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0.5 & 0 & 0 & 0 \\ 0.5 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

Node ordering here is the obvious A, B, C, D. We use a column-stochastic matrix (i.e., columns sum to 1) here, but could also have used a mathematically equivalent row-stochastic matrix (i.e., rows sum to 1).

Now we factor in the teleportation:

* We start with $P = 0.85 * M$, i.e., we multiply each entry in M by $1 - 0.15 = 0.85$.

* We then add in $0.15 / 4 = 0.0375$ to each entry of P , to account for the teleportation. Teleportations (x, x) are allowed.

This gives us:

$$P = \begin{bmatrix} 0.0375 & 0.0375 & 0.8875 & 0.0375 \\ 0.4625 & 0.0375 & 0.0375 & 0.0375 \\ 0.4625 & 0.8875 & 0.0375 & 0.8875 \\ 0.0375 & 0.0375 & 0.0375 & 0.0375 \end{bmatrix}$$

- (b) Evolving x_0 a single step is done by $x_1 = Px_0$ (with a column-stochastic P) or by $x_1 = x_0P$ (with a row-stochastic P) as P already encodes the teleportation. Carrying out this multiplication gives $x_1 = [0.25, 0.14375, 0.56875, 0.0375]^T$.

(c) Considering how the iterations redistribute the total probability mass between the nodes, we can reason:

- (i) C receives incoming links from all of A, B and D. Also C is part of the small cycle $C \rightarrow A \rightarrow C$ which helps retain and circulate probability mass between A and C. These incoming links (especially from nodes that themselves are reachable from the cycle) concentrate more random-walk mass on C, so C will tend to have the largest PageRank among the four nodes.
- (ii) D has no incoming links from the other nodes (it only has an outgoing link to C). That means in the link-following part of the process D does not receive mass from the graph, its only incoming mass is from teleportation. Thus D's PageRank will be small, equal mostly to the teleportation baseline. Teleportation guarantees that its PageRank won't be zero, but D remains the weakest of all nodes in terms of PageRank unless links into D are added.